

7. Multimodal Visual Understanding + SLAM Navigation + Visual Line Following

1. Course Content

1. Learn to use compound function cases such as the robot's visual understanding, visual line following, and SLAM navigation.
2. Learn the new key source code appearing in the tutorial.
3. **Note: This chapter requires completing the map mapping file configuration according to the previous chapter, Multimodal Visual Understanding + SLAM Navigation.**

2. Preparation

2.1 Content Description

The virtual machine used by default with this product starts the Dify Docker environment automatically at boot, so no additional startup is usually needed.

⚠ The same test instructions will not produce exactly the same reply content from the large model, and will differ slightly from the screenshots in the tutorial. If you need to enhance or reduce the diversity of the large model's replies, refer to **【AI Large Model Development】 - 【RAG Knowledge Base Deployment】** to modify the knowledge base and sample library.

Open the virtual machine terminal, start the Dify environment:

```
sh ~/bringup_dify.sh
```

```
yahboom@yahboom:~$ sh ~/bringup_dify.sh
[+] up 13/13
✓ Container docker-sandbox-1      Running      0.0s
✓ Container docker-web-1          Running      0.0s
✓ Container docker-db_postgres-1  Healthy     22.5s
✓ Container docker-ssrf_proxy-1    Running      0.0s
✓ Container docker-redis-1         Running      0.0s
✓ Container docker-weaviate-1      Running      0.0s
✓ Container docker-worker-1        Running      0.0s
✓ Container docker-api_websocket-1 Running      0.0s
✓ Container docker-plugin_daemon-1 Running      0.0s
✓ Container docker-worker_beat-1   Running      0.0s
✓ Container docker-api-1           Running      0.0s
✓ Container docker-nginx-1         Running      0.0s
✓ Container docker-init_permissions-1 Exited       28.4s
yahboom@yahboom:~$
```

3. Running the Case

3.1 Starting the Program

Open the virtual machine terminal and enter the command to start the underlying proxy + camera topic + navigation:

```
ros2 launch microros_v2_bringup nav2.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup nav2.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-24-11-57-50-009345-yahboom-219629
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [rviz2-1]: process started with pid [219664]
[INFO] [micro_ros_agent-2]: process started with pid [219666]
[INFO] [camera_driver-3]: process started with pid [219668]
[INFO] [robot_state_publisher-4]: process started with pid [219670]
[INFO] [joint_state_publisher-5]: process started with pid [219672]
[INFO] [ekf_node-6]: process started with pid [219674]
[INFO] [component_container_isolated-7]: process started with pid [219682]
[micro_ros_agent-2] [1784865471.046289] info      | UDPv4AgentLinux.cpp | init                | running...          | port: 8090
[robot_state_publisher-4] [INFO] [1784865471.195985237] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-4] [INFO] [1784865471.196067136] [robot_state_publisher]: got segment base_link
[robot_state_publisher-4] [INFO] [1784865471.196074446] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-4] [INFO] [1784865471.196079846] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-4] [INFO] [1784865471.196084086] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-4] [INFO] [1784865471.196088634] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-4] [INFO] [1784865471.196092897] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-4] [INFO] [1784865471.196097021] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-4] [INFO] [1784865471.196101332] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-4] [INFO] [1784865471.196105510] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-5] [INFO] [1784865471.581793865] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic...
[camera_driver-3] [INFO] [1784865471.616683871] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[component_container_isolated-7] [INFO] [1784865471.621621536] [nav2_container]: Load Library: /opt/ros/humble/lib/libmap_server_core.so
[component_container_isolated-7] [INFO] [1784865472.58867350] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::CostmapFilterInfoServer>
[component_container_isolated-7] [INFO] [1784865472.588723330] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::MapSaver>
[component_container_isolated-7] [INFO] [1784865472.588730433] [nav2_container]: Found class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::MapServer>
[component_container_isolated-7] [INFO] [1784865472.588735470] [nav2_container]: Instantiate class: rclcpp_components::NodeFactoryTemplate<nav2_map_server::MapServer>
```

Open another virtual machine terminal and enter the command:

```
ros2 launch multi_brains llm_agent_control.launch.py
```

After initialization is complete, the following content will be displayed:

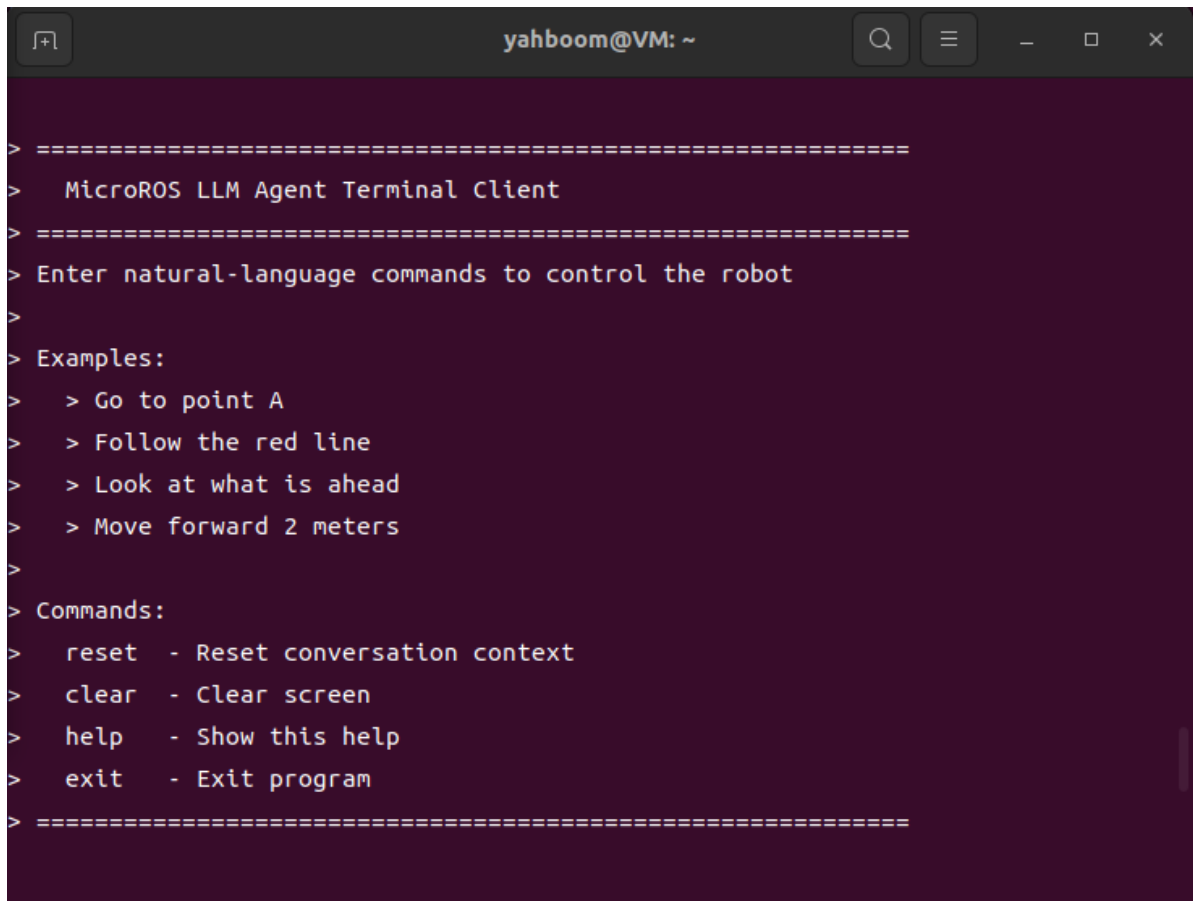
```
yahboom@VM: ~  
G_MODE=False, LANGUAGE=en, CAR_TYPE=ptz  
[llm_agent_node-1] [INFO] [1787128762.059479166] [llm_agent_node]: Initializing  
LLM Agent Node...  
[llm_agent_node-1] [INFO] [1787128762.070369334] [llm_agent_node]: [CONFIG] cmd_  
vel_topic=/cmd_vel  
[llm_agent_node-1] [INFO] [1787128762.077274270] [llm_agent_node]: [CONFIG] imag  
e_cache_path=/home/yahboom/microros_ws/cache/llm_camera.jpg  
[llm_agent_node-1] [INFO] [1787128762.078819208] [llm_agent_node]: [PTZ] startup  
initialization enabled: x=0.0°, y=-45.0°, count=20, interval=0.10s  
[llm_agent_node-1] [INFO] [1787128762.106861388] [llm_agent_node]: [FOLLOW] scan  
_topic=/scan, obstacle_guard=True, buzzer_topic=/beep, buzzer=True  
[llm_agent_node-1] [INFO] [1787128762.107460365] [llm_agent_node]: [DIFY] Initia  
lize client: base_url=http://localhost/v1, api_key=app-SAnX****Zt2E  
[llm_agent_node-1] [INFO] [1787128762.109741022] [llm_agent_node]: [DIFY] Testin  
g connection....  
[llm_agent_node-1] [INFO] [1787128762.130633809] [llm_agent_node]: [TRACK_KCF] S  
ubscribed /tracking_bbox, type=smart_tracker/msg/TrackBox  
[llm_agent_node-1] [INFO] [1787128762.131212772] [llm_agent_node]: LLM Agent Nod  
e initialized successfully  
[llm_agent_node-1] [INFO] [1787128762.133300019] [llm_agent_node]: [DIFY] Connec  
tion successful ✓  
[llm_agent_node-1] [INFO] [1787128764.080740503] [llm_agent_node]: [PTZ] Servo i  
nitialization complete: x=0.0°, y=-45.0°
```

Then follow the process of starting the navigation function for initial localization. After opening the RViz2 visualization interface, click **2D Pose Estimate** in the toolbar at the top, roughly mark the robot's position and orientation on the map. After initial localization, the preparation is complete.



Then open another terminal and start the text dialogue:

```
ros2 run multi_brains terminal_client
```



A terminal window titled 'yahboom@VM: ~' with standard window controls. The terminal output shows the 'MicroROS LLM Agent Terminal Client' interface. It features a header with dashed lines, followed by the title 'MicroROS LLM Agent Terminal Client' and another dashed line. The prompt is 'Enter natural-language commands to control the robot'. Below this, there are 'Examples:' and 'Commands:' sections. The examples include 'Go to point A', 'Follow the red line', 'Look at what is ahead', and 'Move forward 2 meters'. The commands include 'reset' (Reset conversation context), 'clear' (Clear screen), 'help' (Show this help), and 'exit' (Exit program). The terminal ends with a dashed line and a prompt character '>'.

```
> =====
>   MicroROS LLM Agent Terminal Client
>   =====
> Enter natural-language commands to control the robot
>
> Examples:
>   > Go to point A
>   > Follow the red line
>   > Look at what is ahead
>   > Move forward 2 meters
>
> Commands:
>   reset - Reset conversation context
>   clear - Clear screen
>   help  - Show this help
>   exit  - Exit program
> =====
>
```

3.2 Test Cases

Reference test cases are given here. Users can create their own dialogue commands.

- Record the current position, navigate to the tea room and observe whether there is a red line on the ground. If there is a red line, automatically follow the red line. After line following ends, return to the starting point.

3.2.1 Case 1

Enter "Record the current position, navigate to the tea room and observe whether there is a red line on the ground. If there is a red line, automatically follow the red line. After line following ends, return to the starting point." in the terminal. The terminal prints information as follows:

```

> [Sending] Record current position, navigate to pantry to observe if there's a red line on the
ground. If so, perform automatic red-line patrolling. After patrolling, return to starting point.
> [Sent] New commands can be entered; the latest will take precedence.

>
> [Progress] [Decision] Thinking...
>
> [Progress] [Decision] Start executing task. First, record current position as starting point.
>
> [Progress] Decision-level planning: 1. Obtain current position coordinates.
>
> [Progress] [Preparation] Adjusting camera to face directly ahead...
>
> [Progress] [Action 1/1] get_current_pose()
>
> [Progress] [Action 1/1] get_current_pose() Successful ✓
>

```

Then the execution-layer large model will execute according to the decision-layer steps:

The screenshot displays a robotic navigation system interface. On the left, a window titled "LLM Camera" shows a first-person view of a robot in a simulated environment with a red line on the floor. Below this, a "2D Pose Estimate" window shows a top-down view of the robot's position and orientation. On the right, a terminal window shows the system's internal logs, including planning steps, action execution, and feedback.

```

yahboom@yahboom: ~
Planning Layer: 1. Capture current image and provide feedback
[llm_agent_node-1] [INFO] [1784881406.408090249] [llm_agent
Action 1/1] seewhat()
[llm_agent_node-1] [INFO] [1784881406.408408760] [llm_agent
Start capturing current image
[llm_agent_node-1] [INFO] [1784881406.404998573] [llm_agent
ge saved: /home/yahboom/workspaces/MicroROS-V2-ROS2-SRC/mic
g/llm_camera.jpg (640x480)
[llm_agent_node-1] [INFO] [1784881406.410848684] [llm_agent
Displaying saved photo for 2.5 seconds (Qt=xcb)
[llm_agent_node-1] [INFO] [1784881406.411455467] [llm_agent
Image registered for Dify feedback
[llm_agent_node-1] [INFO] [1784881406.412809920] [llm_agent
Action 1/1] seewhat() Success ✓
[llm_agent_node-1] [INFO] [1784881406.532818401] [llm_agent
Picture 1/1 Upload complete: file_id=6-4337-83d4-700341ddib

> [Progress] [Navigation] Going to Point A...
> [Progress] [Navigation] Arrived at Point A ✓
> [Progress] [Action 1/1] navigation(A) Success ✓
> [Progress] [Feedback Decision] Arrived at pantry, now
> [Progress] Planning Layer: 1. Take current image and
> [Progress] [Action 1/1] seewhat()
> [Progress] [Action 1/1] seewhat() Success ✓

```

Reset | Left-Click: Rotate. Middle-Click: Move X/Y. Right-Click/Mouse Wheel: Zoom. Shift: More options.


```
[llm_agent_node-1] [INFO] [1784881384.910535461] [llm_agent_node]: [PROGRESS] [Feedback Decision] Position recorded as point zero, navigating to the pantry now
[llm_agent_node-1] [INFO] [1784881384.912240064] [llm_agent_node]: [PROGRESS] Planning Layer Planning: 1. Navigate to Point A
[llm_agent_node-1] [INFO] [1784881384.913526747] [llm_agent_node]: [PROGRESS] [Action 1/1] navigation(A)
[llm_agent_node-1] [INFO] [1784881384.914229851] [llm_agent_node]: [NAVIGATION] Started standard navigation: A
[llm_agent_node-1] [INFO] [1784881384.915130945] [llm_agent_node]: [NAV] Target coordinate: pos=(-1.058, 0.621, 0.000), ori=(0.000, 0.000, -0.470, 0.882)
[llm_agent_node-1] [INFO] [1784881384.923707642] [llm_agent_node]: [NAV] Navigation goal accepted: 'A'
[llm_agent_node-1] [INFO] [1784881384.925372805] [llm_agent_node]: [NAV] Waiting for arrival at 'A' (Timeout 300.0s)...
[llm_agent_node-1] [INFO] [1784881384.925651668] [llm_agent_node]: [PROGRESS] [Navigation] Going to Point A...
[llm_agent_node-1] [INFO] [1784881401.520472498] [llm_agent_node]: [NAV] result_callback: status=4, result_type=NavigateToPose_Result
[llm_agent_node-1] [INFO] [1784881401.521140210] [llm_agent_node]: [NAV] Navigation complete: status=4
[llm_agent_node-1] [INFO] [1784881401.521472205] [llm_agent_node]: [NAV] Successfully arrived at 'A' ✓
[llm_agent_node-1] [INFO] [1784881401.521822847] [llm_agent_node]: [PROGRESS] [Navigation] Arrived at Point A ✓
[llm_agent_node-1] [INFO] [1784881401.522150210] [llm_agent_node]: [PROGRESS] [Action 1/1] navigation(A) Success ✓
[llm_agent_node-1] [INFO] [1784881406.399312416] [llm_agent_node]: [PROGRESS] [Feedback Decision] Arrived at the pantry, now obtaining the current image to observe ground
[llm_agent_node-1] [INFO] [1784881406.399736915] [llm_agent_node]: [PROGRESS] Planning Layer Planning: 1. Capture current image and provide feedback
[llm_agent_node-1] [INFO] [1784881406.400090249] [llm_agent_node]: [PROGRESS] [Action 1/1] seewhat()
[llm_agent_node-1] [INFO] [1784881406.400408760] [llm_agent_node]: [SEEMHAT] Started capturing current image
[llm_agent_node-1] [INFO] [1784881406.404990573] [llm_agent_node]: [SEEMHAT] Image saved: /home/yahboon/workspace/MicroROS-V2-ROS2-SRC/microros_car_v2_ws/config/llm_camera.jpg (640x480)
[llm_agent_node-1] [INFO] [1784881406.410848684] [llm_agent_node]: [SEEMHAT] Displaying saved photo for 2.5 seconds (Qt=xcv)
[llm_agent_node-1] [INFO] [1784881406.411455467] [llm_agent_node]: [SEEMHAT] Image registered as Dify feedback
[llm_agent_node-1] [INFO] [1784881406.412809920] [llm_agent_node]: [PROGRESS] [Action 1/1] seewhat() Success ✓
[llm_agent_node-1] [INFO] [1784881406.532818401] [llm_agent_node]: [DIFY] Feedback Image 1/1 Upload complete: file_id=8fe4235d-f956-4337-83d4-700341dd1b2f, HTTP 201
[llm_agent_node-1] [INFO] [1784881417.344943947] [llm_agent_node]: [PROGRESS] [Feedback Decision] The image shows clear red lines on the ground, now starting the red line
[llm_agent_node-1] [INFO] [1784881417.345396844] [llm_agent_node]: [PROGRESS] Planning Layer Planning: 1. Follow red line forward
[llm_agent_node-1] [INFO] [1784881417.345776338] [llm_agent_node]: [PROGRESS] [Action 1/1] follow_line(red)
[llm_agent_node-1] [INFO] [1784881417.346109558] [llm_agent_node]: [FOLLOW] Started line following (car_type=ptz, color=red)
[llm_agent_node-1] [INFO] [1784881417.346503126] [llm_agent_node]: [PROGRESS] [Line Following] Adjusting camera line following angle...
[llm_agent_node-1] [INFO] [1784881417.848813478] [llm_agent_node]: [PROGRESS] [Line Following] Traveling along the red line path...
[llm_agent_node-1] [INFO] [1784881417.987466661] [llm_agent_node]: [FOLLOW] start colored, hsv=(0, 118, 99), (11, 205, 187)), hsv_file=/home/yahboon/workspaces/MicroROS-V2-ROS2-SRC/microros_car_v2_ws/src/multi_brains/config/color_hsv.txt
```

After the robot completes the task, it enters a waiting state. At this time, instructions will continue to be passed to the execution-layer large model, and the conversation history will be retained. You can continue the dialogue in the terminal and input the "/reset" command to let the robot end the current task cycle and start a new task cycle.

```
[llm_agent_node-1] [INFO] [1784867518.113324588] [llm_agent_node]: [INPUT] — Received user input #3: 'reset'
[llm_agent_node-1] [INFO] [1784867518.114754858] [llm_agent_node]: [PROGRESS] [Decision] Thinking...
[llm_agent_node-1] [INFO] [1784867522.880420933] [llm_agent_node]: [ACTION] finishtask: Task cycle finished
[llm_agent_node-1] [INFO] [1784867522.881048600] [llm_agent_node]: [RESULT] System reset, awaiting new commands
[llm_agent_node-1] [INFO] [1784867522.881456002] [llm_agent_node]: [INPUT] — Instruction #3 processing complete, total time: 4.77s —
```

4. Source Code Analysis

This compound case chains together chassis motion, current position recording, SLAM navigation, visual understanding, and visual line following. In the current version, `llm_agent_node.py` schedules uniformly, executing only one action per round, and after the action is completed, the result is fed back to Dify, which decides the next step.

Current relevant source code paths:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
~/microros_ws/src/multi_brains/multi_brains/controllers/motion_controller.py
~/microros_ws/src/multi_brains/multi_brains/controllers/navigation_controller.py
~/microros_ws/src/multi_brains/multi_brains/controllers/follow_line_controller.py
```

Possible action sequence:

```
get_current_pose()
navigation(target)
seewhat()
follow_line(green)
navigation(zero)
finishtask()
```

Key points:

- `get_current_pose()` records the current position as the runtime dynamic point `zero`.
- `navigation(target)` uses the target point coordinates in `config/map_mapping.yaml`.
- `seewhat()` uploads the current frame to Dify to determine whether there is a specified color line on the ground.
- `follow_line(color)` reads the color calibration values in `config/color_hsv.txt` and controls the chassis line following.
- If a new instruction is input at any stage, the current action is preempted by the latest instruction, and navigation, line following, and chassis actions are stopped as soon as possible.